

K-Nearest Neighbors (KNN)

What is KNN?

Definition: K-Nearest Neighbors (KNN) is a simple, non-parametric, and instance-based learning algorithm used for classification and regression. It classifies a data point based on how its neighbours are classified.

Explanation: Imagine you've just moved to a new estate in Nairobi, say, **Kilimani**. You don't know anyone yet, but you're trying to figure out what kind of vibe this place has. You want to know: *is this estate more "quiet-family" type or "party-all-weekend" type?*

So you do what any Kenyan would do: you **observe your neighbors**.

Here's how KNN plays out in this situation:

1. **You (the new person)** are the **data point to be classified**.
You don't yet belong to any "group" (quiet family or party-all-weekend).
2. **All your neighbors**, the people already living around you, are **labeled data points**.
You know their type because you've seen their lifestyle:
 - Mama Njeri with her three kids and sukuma wiki garden = *quiet family*.
 - Kevin and his boys, who host FIFA tournaments till 3 a.m. = *party-all-weekend*.
3. Now, you decide to look at the **K closest neighbors** to you.
Maybe you choose $K = 5$, meaning you'll check out your **five nearest households**.
4. You count how many among those 5 are "quiet" vs "party."
Let's say 3 are quiet and 2 are party types.
5. **The majority wins**.
Since most of your nearest neighbors are quiet-family types, KNN concludes *you probably fit into a quiet-family neighborhood*.
So you'd be classified as **"quiet-family"** too.

In data science terms:

- Each "neighbor" is a data point with known labels.
- The "distance" between you and them could be calculated using math (like Euclidean distance), but conceptually, it's just *how similar* you are to them.
- "K" decides how many neighbors to consider when voting.

What is K?

K is the number of neighbors we consult.

If **K = 1**, we copy the label of the single closest point.

What does “copy the label” mean?

A **label** is just the answer attached to a past example.

- If we're classifying fruit, labels could be “**orange**” or “**lemon.**”
- If we're predicting loan risk, labels could be “**default**” or “**not default.**”

So “copy the label” just means: **use the same answer as the nearest example.**

You look for the single closest example in your training data.

- **If the closest fruit in your saved examples is labeled “orange”**
- **Then your new fruit is predicted as “orange”**

It's like asking one of your closest friends for advice and doing exactly what they say.

If **K = 5**, we look at the closest 5 points and take a vote (**classification**) or average (**regression**).

You look for the **5 closest** examples.

For a classification problem (category answer)

You take a **vote**.

Example (5 closest neighbors):

- 3 are labeled **orange**
- 2 are labeled **lemon**

The majority wins → predict **orange**.

It's like asking **5 friends** and going with what **most** of them say.

For a regression problem (numerical answer)

You take an **average**.

Example (5 closest houses with known prices):

- 4.8M, 5.0M, 5.1M, 4.9M, 5.2M

Average $\approx$ 5.0M $\rightarrow$ that becomes your prediction.

It's like checking **5 similar houses** and estimating your house price as their **average**.

Now, if you had moved to Westlands, your nearest five neighbors might be all “party-all-weekend” types, so the algorithm would label you accordingly. KNN simply reflects the **local pattern** around a data point.

It's basically *“show me your neighbors, and I'll tell you who you are.”*

What KNN does

KNN is a memory-based method. It does not "learn"; instead, it remembers the training data. So we can also consider it Lazy to some degree.

It's like a **very observant person**. Instead of learning rules or doing math, it just **remembers all the past examples**.

Real-Life Example:

Say you want to predict someone's taste in movies. You look at their 5 closest friends. If 3 out of 5 love horror, you predict that this person will like horror too.

"Show me who your " K nearest " friends are and I'll tell you who you are?"



The "Training" Process of KNN:

1. **It just stores the data:**

When you “train” a KNN model, you’re not really teaching it anything; you’re just feeding it the data and saying, “Hey, remember this!”

That means KNN stores:

- The input features (like age, income, height, whatever describes the data)
- The correct label or outcome (like “buy”, “don’t buy”, or a category like “cat” or “dog”)

2. **No actual learning:**

There’s no calculation or model building during training. It’s like a memory bank; it just keeps all the data safe for later.

When It Makes a Prediction (after training):

When you give KNN a new example, it:

1. It looks at all the **past examples**.
 2. **Measures** how close each one is to the new example.
 3. Picks the **K closest** ones.
 4. **Takes a vote from those K to guess the correct answer.** Based on whether you are dealing with a classification issue or a regression problem
-

In Simple Terms:

Training KNN is like **collecting labeled examples and saving them**. It doesn’t try to figure anything out at that time; it just waits until you ask it to make a decision, then it looks around and copies the majority.

KNN Distance Parameter

Definition: The distance parameter in KNN determines how the similarity between data points is measured. The most common distance metrics are **Euclidean distance** and **Manhattan distance**.

Explanation:

- **Euclidean Distance:** The straight-line distance between two points in Euclidean space.
 - **Formula:** $d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$
 - **Example:** Measuring the direct distance between two houses on a map.
- **Manhattan Distance:** The sum of the absolute differences between the coordinates of the points.
 - **Formula:** $d(x, y) = \sum_{i=1}^n |x_i - y_i|$
 - **Example:** Measuring the distance between two points in a city grid where you can only move along streets and avenues.

Example: Consider a recommendation system for movies. To find similar movies to recommend, you might use Euclidean distance to measure the similarity between movies based on their features, such as genre, rating, and length.

Note: some categorical features need to be turned into numerical features first through encoding. (one hot encoding or Label encoding)

A. Real-World Example: Finding the Nearest Store using Euclidean Distance

Imagine you are in a city and want to find the nearest grocery store. You have a list of store locations and your current location, and you want to find which store is closest to you. **(as-the-crow-flies). Does not take into account pathways and roads.**

How It Works:

1. **Locations as Points:** Think of your location and each store's location as points on a map.
2. **Coordinates:** Each location can be represented by coordinates (like (x, y) on a 2D grid). For simplicity, let's say **Westside Towers** is at **(2, 3)** and the stores are:
3. The Mall **(5, 1)**, Westgate **(4, 1)**, and Sarit Centre **(4, 5)**.

Check the distances (using Euclidean distance)

Euclidean distance from (x_1, y_1) to (x_2, y_2) is:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Distance to each Shopping Mall:

- Distance to Store 1: **3.61** - The Mall

Your location: Westside Towers = (2, 3)

1) The Mall (5, 1)

$$\sqrt{(5 - 2)^2 + (1 - 3)^2} = \sqrt{3^2 + (-2)^2} = \sqrt{9 + 4} = \sqrt{13} \approx 3.61$$

✓ Your 3.61 is correct.

- Distance to Store 2: **2.83** - Westgate
- Distance to Store 3: **2.83** - Sarit Centre

So the nearest is actually a **tie between Westgate and Sarit Centre** (both equally close).

Other Real-World Uses:

- 1. Recommendation Systems (like Netflix or Spotify)**

Think of Euclidean distance as measuring *how alike two people's tastes are*.

If you and I both watch "Money Heist," "Snowfall," and "Queen of the South," the *distance* between our preferences is small, meaning we're similar.

So Netflix says, "*Since Pete liked what this other user liked, he'll probably enjoy 'Narcos' too.*"

- 2. Image Processing (like Google Photos)**

When you upload pictures, Google Photos groups similar faces together.

It doesn't "see" faces the way we do; it measures *feature distances* (like eyes, nose, jawline) in a high-dimensional space.

The closer two faces are in Euclidean distance, the more likely they belong to the same person.

It's basically geometry doing detective work.

- **3. Location-Based Apps (like Uber or Bolt)**

Euclidean distance measures how far you are from your driver — “Driver is 2.3 km away.”

Behind the scenes, it’s just computing the straight-line distance between your coordinates and theirs.

Of course, roads aren’t straight, but Euclidean distance gives the *rough* spatial idea before adjusting for the route.

- **4. Sports and Fitness Trackers**

When your smartwatch says you ran 5 km, it’s adding up tiny Euclidean distances between GPS points as you move.

Each pair of latitude/longitude readings forms a little triangle — and the watch sums all those tiny straight-line steps to find your total distance.

So, your morning jog is basically a math problem in disguise.

- **5. Fraud Detection in Finance**

Banks use Euclidean distance to spot transactions that *don’t fit your usual pattern*.

If you always spend around KSh 500 on food in Nairobi and suddenly make a KSh 50,000 transaction in Dubai, the “distance” between that new behavior and your usual one is huge.

Red Flags raised. Possible fraud.

- So, in plain terms, **Euclidean distance** = how close or far two things are, based on their characteristics.

It’s the world’s simplest but most widely used “**similarity detector**.”

B. Real-World Example: Navigating City Streets using Manhattan distance

Picture this:

You’re at **point A** — say, *your house in South B*.

You want to go to **point B** — *a mall in the CBD*.

Now, if you were a bird, you’d just fly straight there, that’s **Euclidean distance** (the shortest, diagonal line).

But you’re not a bird. You’re a Kenyan dealing with **roads, roundabouts, and matatus blocking the lane**.

You can only move **along the streets**, either east–west or north–south, like you’re navigating a city grid.

That's what **Manhattan distance** measures.

It's named after **Manhattan**, New York, because its streets form neat blocks where you can't cut diagonally, just like Nairobi's downtown roads.

How It Works

Each location is a **point on a grid**, with coordinates like (x, y) .

Let's say:

- You are at $(2, 3)$ → your home.
- The mall is at $(5, 1)$.

To get from $(2, 3)$ to $(5, 1)$, you must move:

- From $x = 2$ to $x = 5$ → **3 blocks east**
- From $y = 3$ to $y = 1$ → **2 blocks south**

You can't cut across; you must walk the grid.

So the **Manhattan distance** is the total number of blocks:

$3 + 2 = 5$ units.

In formula terms:

In formula terms:

$$\text{Manhattan Distance} = |x_1 - x_2| + |y_1 - y_2|$$

Other Real-World Uses:

- **Delivery Apps:** Riders don't drive in straight lines through buildings — they follow roads, so Manhattan distance often models **real-world travel paths** better than Euclidean.
- **Robots in Warehouses:** When robots move between aisles (like at Amazon), they can only move horizontally or vertically. They use Manhattan distance to plan routes.
- **Machine Learning:** Some algorithms (like KNN) may use Manhattan distance instead of Euclidean when data dimensions represent *independent categories* (like number of steps east/west rather than continuous space).

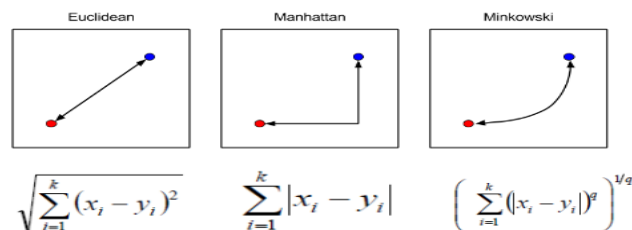
If Euclidean distance is “*as the crow flies*,” then Manhattan distance is “*as the Nairobi driver drives — painfully, block by block*.”

Distance Metric

Common options:

- Euclidean Distance (straight-line)
- Manhattan Distance (block-by-block)

⚠ Choosing the right **K** and distance metric is key!



Classification vs Regression (in plain terms)

KNN Classification

- Output is a category (example: orange vs lemon, default vs non-default).
- The K neighbors vote, majority wins.

KNN Regression

- Output is a number (example: house price).
- The prediction is typically **the average (mean)** of the K neighbors' target values.

KNN Steps

Steps to Implement KNN:

1. **Data Preparation:**
 - **Explanation:** Collect and preprocess the dataset, including normalisation or standardisation if necessary.
 - **Example:** Prepare a dataset of fruits with features like weight, colour, and texture.

2. Choose the Value of K:

- **Explanation:** Select the number of neighbours (k) to consider for classification.
- **Example:** Decide to use $k=3$ for a small dataset.
- **Explanation:** Pick how many nearby data points you want to use when estimating a numeric answer. (For Regression Problems)
- **Example:** Choose **K = 3**. That means when you want to predict a value for a new point, you find its **3 nearest neighbors** and use their **target values** to estimate the answer (usually by taking the **average**).

3. Calculate Distance:

- **Explanation:** Compute the distance between the new data point and all other points in the training set.
- **Example:** Calculate the Euclidean distance between a new fruit and all other fruits in the dataset.

4. Identify Neighbours:

- **Explanation:** Identify the k closest neighbours to the new data point.
- **Example:** Find the three nearest fruits to the new fruit based on distance.

5. Vote for Class Labels:

- **Explanation:** Count the class labels of the k neighbours and assign the most frequent class to the new data point.
- **Example:** If two out of three nearest fruits are apples, classify the new fruit as an apple.

Finding The Optimal K Value in KNN

Definition:

The *optimal K value* is the number of neighbors that gives the best performance for your KNN model in other words, the sweet spot between being too picky and too careless when making predictions.

Explanation:

Choosing the right K is like choosing how many friends to consult before making a decision.

- If you only ask **1 friend (K = 1)**, your result depends entirely on that one person; if they're wrong, you're wrong. That's **overfitting** (too sensitive to noise).
- If you ask **everyone in the estate (K too large)**, the result might be too averaged out, and you lose the unique pattern in your area. That's **underfitting** (too generalized).

So, the goal is to find the **K that balances** these two extremes, where your model is neither too jumpy nor too lazy in decision-making.

Rule of thumb:

Small K = more flexible, more noise-sensitive

Large K = smoother, more stable, but can miss detail

How to Find the Optimal K (Two Methods)

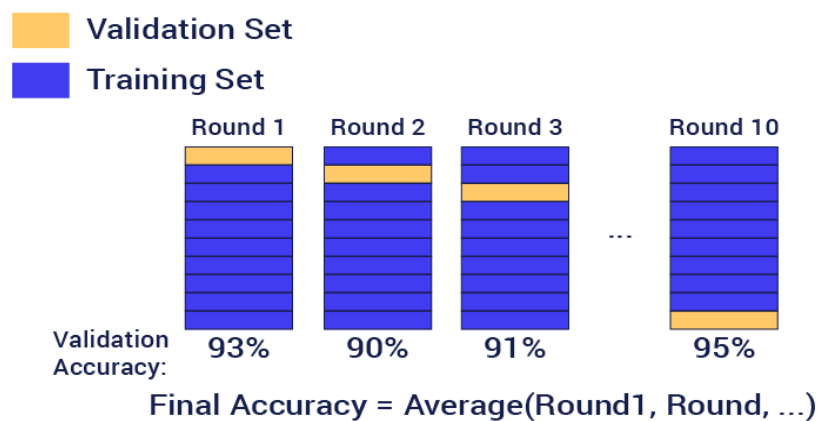
1. Cross-Validation

Use **k-fold cross-validation** to test how the model performs with different K values. You split your dataset into parts (folds), train on some, test on others, and rotate.

Example:

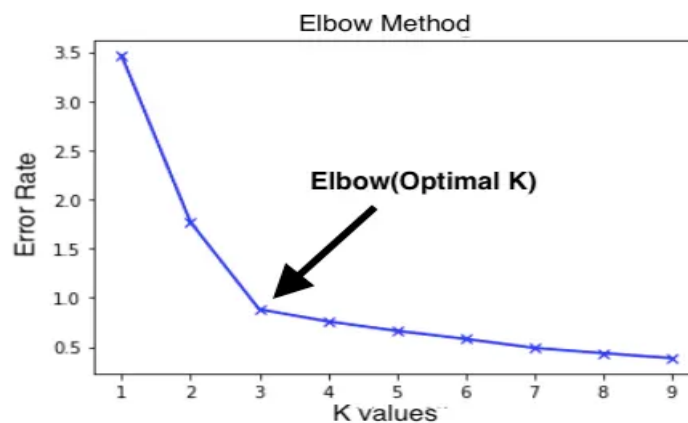
Try K values from 1 to 20 and record accuracy for each.

The K that gives the **highest average accuracy** across folds is your optimal K.



2. Elbow Method

Plot **accuracy vs. K values** on a graph.



At first, accuracy improves as K increases, but after a point, the gains flatten out — forming an “**elbow**” shape on the graph.

Example:

Choose the K where accuracy stops increasing sharply; that’s your elbow point. Beyond that, you’re not gaining much, so it’s not worth the extra complexity.

(Just like adding more friends to a group chat, at first it’s useful, then it just gets noisy.)

In short:

- **Small K → Overfitting (too specific)**
 - **Large K → Underfitting (too general)**
 - **Optimal K → Just right (balanced learning)**
-

Advantages & Disadvantages of the KNN Algorithm

Advantages

- It is very easy to understand and implement
- It is an instance-based learning(lazy learning) algorithm.
- KNN does not learn during the training phase, hence new data points can be added without affecting the performance of the algorithm.
- It is well-suited for small datasets.

Disadvantages

- It fails when variables have different scales.
- It is difficult to choose the K-value.
- It leads to ambiguous interpretations.
- It is sensitive to outliers and missing values.
- Does not work well with large datasets.
- It does not work well with high dimensions

More On KNN

[One Stop for KNN. Nearest Neighbors? How near are these... | by Priyansh Soni | Medium](#)